Laboratory 0b101

Expected submission ON PORTALE DELLA DIDATTICA:
SURNAME_NAME_MATRICOLA_lab_05.zip

Including (in case you have projects with the same name, you can prepend lab_5 to them):
- program_1
- lab_05.pdf (complete this file and export it as PDF)

Submission deadline: 03/11/2026

Exercise 1:

1) Write a new assembly program (program_1) that implements the following C code:

```
for (i = 0; i < 100; i++) {
      v4[i] = (v1[i] / v2[i]) + v3[i];
      if (int(i/3) == 0) {
            v5[i] = v4[i] * v3[i];

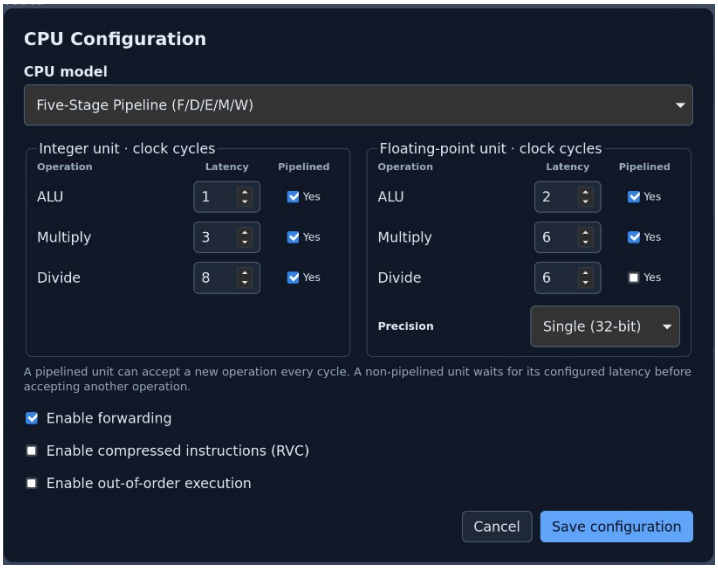
      } else if (int(i/3) == 1) {
            v5[i] = v4[i] / v3[i];

      } else {
            v5[i] = v4[i] + v3[i];
      }
}
```

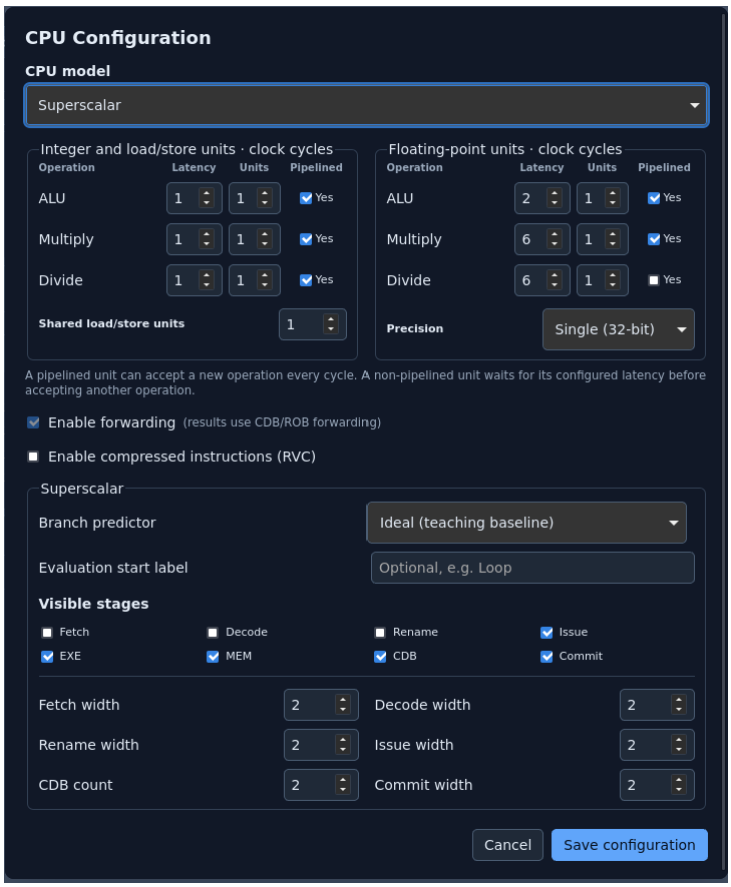
Considering v2 and v3 as arrays of 100 floating point values, always different from zero

2) Simulate the program using the two CPU configurations shown below. Configuration 1 uses an In-Order CPU, while Configuration 2 uses an Superscalar CPU. In the second case the Ideal BPU is active (i.e., assume that every branch prediction is correct).

Configuration 1)



Configuration 2)



Run both configurations and record the resulting CPI (Clocks Per Instruction) for the In-Order and the Superscalar CPU in the table below:

	program_1
CPI In-Order CPU	2.15
CPI Superscalar CPU	0.69

Compare the two results: which architecture performs better, and why does this happen? Justify your answer.

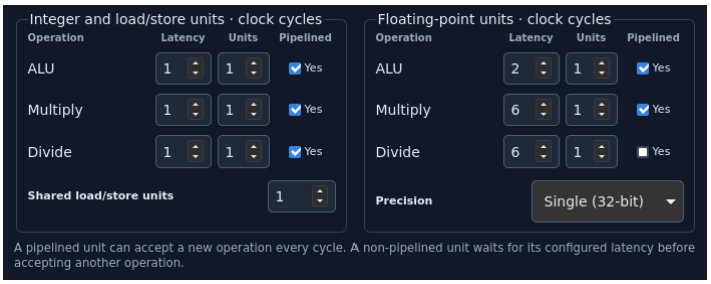
Answer:
Here we can see that the Superscalar processor performs better than a simple pipeline processor because in the superscalar processor the architecture dynamically detects dependencies for example data dependency on the memory and can do at run time register renaming. Also instructions in one clock cycle 2 instruction can be issue and we have several execution unit (integer unit, floating point unit) that working in parallel. This architecture also guaranties precise exceptions

By hand, trace the first three iterations of the loop on the Superscalar CPU and fill in the table below, writing the clock cycle at which each instruction passes through every pipeline stage (Issue, EXE, MEM, CDB, COMMIT):

# iteration		Issue	EXE	MEM	CDB x2	COMMIT x2
1	flw f1, 0(t0)	1	2	3	4	5
1	flw f2, 0(t1)	1	3	4	5	6
1	flw f3, 0(t2)	2	4	5	6	7
1	fdiv.s f4, f1, f2	2	6		7	8
1	fadd.s f4, f4, f3	3	8		9	10
1	fsw f4, 0(t3)	3	4			10
1	div x3, x1, x2	4	5		6	11
1	beqz x3, case_0	5	7			11
1	fmul.s f5, f4, f3	6	10		11	12
1	fsw f5, 0(t4)	6	7			12
1	addi t0, t0, 4	7	8		9	13
1	addi t1, t1, 4	7	9		10	13
1	addi t2, t2, 4	8	10		11	14
1	addi t3, t3, 4	8	11		12	14
1	addi t4, t4, 4	9	12		13	15
1	addi x1, x1, 1	9	13		14	15
1	bne x1, x2, loop	10	15			16
2	flw f1, 0(t0)	11	12	13	14	16
2	flw f2, 0(t1)	11	13	14	15	17
2	flw f3, 0(t2)	12	14	15	16	17
2	fdiv.s f4, f1, f2	12	15		16	18
2	fadd.s f4, f4, f3	13	17		18	19
2	fsw f4, 0(t3)	13	14			19
2	div x3, x1, x2	14	15		16	20
2	beqz x3, case_0	15	17			20
2	fmul.s f5, f4, f3	16	19		20	21
2	fsw f5, 0(t4)	16	21			22
2	addi t0, t0, 4	17	18		19	22
2	addi t1, t1, 4	17	19		20	23
2	addi t2, t2, 4	18	20		21	23
2	addi t3, t3, 4	18	21		22	24
2	addi t4, t4, 4	19	22		23	24
2	addi x1, x1, 1	19	23		24	25
2	bne x1, x2, loop	20	25			26
3	flw f1, 0(t0)	21	22	23	24	26
3	flw f2, 0(t1)	21	23	24	25	27
3	flw f3, 0(t2)	22	24	25	26	27
3	fdiv.s f4, f1, f2	22	26		27	28
3	fadd.s f4, f4, f3	23	28		29	30
3	fsw f4, 0(t3)	23	24			30
3	div x3, x1, x2	24	25		26	31
3	beqz x3, case_0	25	27			31
3	fmul.s f5, f4, f3	26	30		31	32
3	fsw f5, 0(t4)	26	27			32
3	addi t0, t0, 4	27	28		29	33
3	addi t1, t1, 4	27	29		30	33
3	addi t2, t2, 4	28	30		31	34
3	addi t3, t3, 4	28	31		32	34
3	addi t4, t4, 4	29	32		33	35
3	addi x1, x1, 1	29	33		34	35
3	bne x1, x2, loop	30	35			36

Exercise 2: Architectural Changes

In this exercise you will study how the amount of execution resources affects performance. Starting again from the Superscalar configuration of Exercise 1 (with the Ideal BPU), change the number of Integer Load/Store Units (LSU) from 1 to 2. For each configuration, record the CPI in a table and explain what happens: identify which resource becomes the bottleneck, and describe how adding LSUs changes the number of structural stalls and therefore the overall performance. Comment on whether increasing the number of units improves the CPI.



	program_1
Initial Configuration	0.67
CPI 2 LSUs	0.67

Answer:
In my case increasing the number of Load/Store unit does not change or improves the CPI it remain constant but on my clock cycle output I can see that having 2 LSU makes 2 Load or Store institution tu execute in a same clock cycle if the is know memory dependency example
flw f1, 0(t0)
flw f2, 0(t1)
so issued , execution, commit at the same clock cycles

(OPTIONAL) Exercise 3: Branch Prediction Unit (BPU)

Using the Superscalar configuration from Exercise 1, study how the Branch Prediction Unit (BPU) affects performance. Run program_1 first with the Ideal BPU (the teaching baseline, in which every prediction is assumed correct) and then with the Local Counter BPU.

Report for both the CPI and explain why they differ, if so.

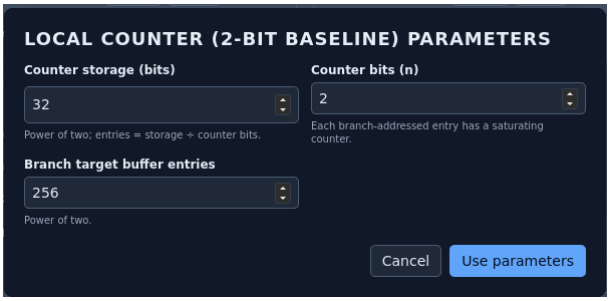
	program_1
CPI Ideal BPU	0.67
CPI Local Counter BPU	0.71

Answer:
This is due to the fact that at the beginning in the BPU keeps telling the fetch unit to continue fetching instructions and at after execution of the branch instruction by the processor the BPU notice it has did a wrong prediction so all the wrong instruction are all removed from the instruction queue, reservation stations, reordered buffer and fetching starts in the proper way. All this mechanism reduces the performance hence that is why we have a CPI of 0.71 compared to the ideal CPI of 0.67 with all ideal prediction but this is not too bad because we have the reordered buffer that helps use not restart the program from scratch because he has already committed some instruction hence it guaranties use with a precise exception.

Starting from the Local Counter BPU, perform the following two experiments, changing one parameter at a time while keeping the other fixed:

a) vary the number of counter bits from 2 to 4 while keeping the size of the prediction table constant.

b) vary the number of counters in the history table from 2 to 8 while keeping the number of counter bits constant. For each setting, record the CPI of the Superscalar CPU.



For every configuration, report the CPI in a table and explain what changes during execution: how the prediction accuracy affects the number of mispredictions, and therefore the pipeline flushes and stalls, and how this is reflected in the CPI. Finally, compare all your results against the Ideal BPU baseline and comment on how close the Local Counter BPU can get to it.

	program_1
Initial Configuration	0.71
CPI Counter Bits 2-> 8	1.99
CPI Counter storage 32 -> 64	0.71

Answer: